

A Technical Introduction to K-means clustering

$$J = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

Section 1

$|S_i|$ is the size of S_i , and $\|\cdot\|$ is the usual L2 norm. This is equivalent to minimizing the pairwise squared deviations of points in the same cluster:

Section 2

n is the number of d -dimensional vectors (to be clustered) k the number of clusters i the number of iterations needed until convergence. On data that does have a clustering structure, the number of iterations until convergence is often small, and results only improve slightly after the first dozen iterations. Lloyd's algorithm is therefore often considered to be of "linear" complexity in practice, although it is in the worst case superpolynomial when performed until convergence.

Section 3

Hartigan–Wong method Hartigan and Wong's method provides a variation of k-means algorithm which progresses towards a local minimum of the minimum sum-of-squares problem with different solution updates. The method is a local search that iteratively attempts to relocate a sample into a different cluster as long as this process improves the objective function. When no sample can be relocated into a different cluster with an improvement of the objective, the method stops (in a local minimum). In a similar way as the classical k-means, the approach remains a heuristic since it does not necessarily guarantee that the final solution is globally optimum. Let $\phi(S_j)$ be the individual cost of S_j defined by $\sum_{x \in S_j} (x - \mu_j)^2$, with μ_j the center of the cluster.

Section 4

In the worst-case, Lloyd's algorithm needs $i = 2^{\Omega(\sqrt{n})}$ iterations, so that the worst-case complexity of Lloyd's algorithm is superpolynomial. Lloyd's k-means algorithm has polynomial smoothed running time. It is shown that for arbitrary set of n points in $[0, 1]^d$, if each point is independently perturbed by a normal distribution with mean 0 and variance σ^2 , then the expected running time of k-means algorithm is bounded by $O(n^{34} k^{34} d^8 \log^4(n) / \sigma^6)$

$O(n^{34} k^{34} d^8 \log^4(n) / \sigma^6)$, which is a polynomial in n , k , d and $1/\sigma$. Better bounds are proven for simple cases. For example, it is shown that the running time of k-means algorithm is bounded by $O(d n^4 M^2)$ for n points in an integer lattice $\{1, \dots, M\}^d$. Lloyd's algorithm is the standard approach for this problem. However, it spends a lot of processing time computing the distances between each of the k cluster centers and the n data points. Since points usually stay in the same clusters after a few iterations, much of this work is unnecessary, making the naïve implementation very inefficient. Some implementations use caching and the triangle inequality in order to create bounds and accelerate Lloyd's algorithm.

Section 5

argmin $S \sum_{i=1}^k \sum_{x \in S_i} \|x - \mu_i\|^2$.

Section 6

Shared Memory Model (OpenMP): This approach divides the work between CPU threads for the reassignment step of the process. These threads independently calculate distances to centroids. Partial sums and counts from each thread are aggregated to compute the new centroids for the next iteration. GPU Parallelization (OpenACC): Unlike the OpenMP implementation, the OpenACC program's parallel directive is called at each necessary step instead of at the very beginning. This exploits the SIMT (Single Instruction, Multiple Threads) architecture of GPUs to perform thousands of distance calculations simultaneously. We see that in both parallel versions; efficiency is greatly increased for large datasets. This speedup is significant in the OpenACC implementation, due to the massively parallel nature of GPU architectures.

Section 7

The objective function in k-means is the WCSS (within cluster sum of squares). After each iteration, the WCSS monotonically decreases, giving a nonnegative monotonically decreasing sequence. This guarantees that the k-means always converges, but not necessarily to the global optimum. The algorithm has converged when the assignments no longer change or equivalently, when the WCSS has become stable. The algorithm is not guaranteed to find the optimal cluster assignment. The algorithm is often presented as assigning objects to the nearest cluster by distance. Using a different distance function other than (squared) Euclidean distance may prevent the algorithm from converging. Various modifications of k-means such as spherical k-means and k-medoids have been proposed to allow using other

distance measures.

Section 8

Assignment step Hartigan and Wong's method starts by partitioning the points into random clusters $\{S_j\}_{j \in \{1, \dots, k\}}$. Update step Next it determines the $n, m \in \{1, \dots, k\}$ and $x \in S_n$ for which the following function reaches a maximum $\Delta(m, n, x) = \phi(S_n) + \phi(S_m) - \phi(S_n \setminus \{x\}) - \phi(S_m \cup \{x\})$. For the x, n, m that reach this maximum, x moves from the cluster S_n to the cluster S_m . Termination The algorithm terminates once $\Delta(m, n, x)$ is less than zero for all x, n, m . Different move acceptance strategies can be used. In a first-improvement strategy, any improving relocation can be applied, whereas in a best-improvement strategy, all possible relocations are iteratively tested and only the best is applied at each iteration. The former approach favors speed, whether the latter approach generally favors solution quality at the expense of additional computational time. The function Δ used to calculate the result of a relocation can also be efficiently evaluated by using equality